

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TECHNICAL PRESENTATION

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	10% of CIA	Total Marks:	100 (1 Question × 100 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	S.Mahammed Nayeem	Reg. No.:	192425294
Section / Batch:	2024 - AI &ML	Date:	24-07-2026

Instructions to Students

- Answer the given question through a technical presentation. Total Marks: 100.
- Clearly define the problem and explain the concept of algorithm growth functions.
- Present comparative analysis using appropriate representations such as graphs, tables, or charts.
- Use suitable tools (PowerPoint, visualization tools, or software) to support your explanation.
- Ensure technical accuracy, logical flow, and proper interpretation of results.
- Maintain clarity in communication, proper slide organization, and professional delivery.
- Time management, confidence, and audience engagement will be considered during evaluation.

Question Type	Focus Area	Questions	Marks
Technical Presentation	Analyze and compare growth functions using mathematical techniques and asymptotic concepts and with graphical and visual interpretation	Q1	100
GRAND TOTAL:		100 Marks	

SECTION A – Technical Presentation

Q1. Visualization-Based Technical Presentation on Growth Functions

Analyze growth functions $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$ using visualization tools in a technical presentation format. Clearly define the purpose of comparing these functions for understanding algorithm efficiency. Use graphs, tables, or charts to show how each function behaves as the input size increases. Explain which functions remain practical for large-scale problems and justify your reasoning. Present the conclusions clearly and professionally by linking growth behavior to algorithm selection.

Code of Conduct

I S.Mahammed Nayeem (192425294) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Mahammed Nayeem

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Marks Evaluation:

Question No.	Technical Content Accuracy (30%)	Problem Identification (25%)	Use of Tools (20%)	Communication (15%)	Professionalism (10%)	Total Marks (100)
Q1	4	4	4	2	3	90
Overall Total (100)						90

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1.Visualization-Based Algorithm Efficiency Analysis

Aim

To analyze the efficiency of algorithms using visualization-based analysis by comparing the growth of different time complexities for increasing input sizes. The study helps understand how algorithms perform as the amount of input data increases and identifies the most suitable algorithm for solving large-scale problems efficiently.

Introduction

Algorithm efficiency is the measure of the computational resources required by an algorithm to solve a problem. The two primary resources considered are execution time and memory usage. Time complexity describes how the running time of an algorithm increases with the input size (n). Visualization-based analysis represents these growth patterns using tables or graphs, making it easier to compare different algorithms and understand their scalability.

Input Assumptions

- Input size is represented by n .
- All algorithms solve the same computational problem.
- The execution environment (processor speed, memory, compiler optimizations) is assumed to be identical.
- Comparison is based on theoretical time complexity rather than actual execution time.
- The best algorithm is determined by observing how slowly its execution time grows as n increases.

Time Complexity Comparison

Algorithm Type	Time Complexity	Growth Rate	Efficiency
Constant	$O(1)$	No growth	Excellent
Logarithmic	$O(\log n)$	Very Slow	Very High
Linear	$O(n)$	Proportional	Good
Linearithmic	$O(n \log n)$	Slightly Faster than Linear	Efficient
Quadratic	$O(n^2)$	Rapid	Moderate

Cubic	$O(n^3)$	Very Rapid	Poor
Exponential	$O(2^n)$	Extremely Rapid	Very Poor

Growth Trend Table

Input Size (n)	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
10	1	3	10	33	100	1,000	1,024
20	1	4	20	86	400	8,000	1,048,576
40	1	5	40	213	1,600	64,000	1.10×10^{12}
80	1	6	80	506	6,400	512,000	1.21×10^{24}
160	1	7	160	1,172	25,600	4,096,000	1.46×10^{48}

Observation

- The execution time of $O(1)$ remains constant regardless of the input size.
- $O(\log n)$ increases very slowly, making it ideal for searching algorithms such as Binary Search.
- $O(n)$ grows linearly with the input size and performs well for sequential processing.
- $O(n \log n)$ is commonly observed in efficient sorting algorithms such as Merge Sort and Quick Sort.
- $O(n^2)$ grows rapidly and is suitable only for small datasets.
- $O(n^3)$ becomes inefficient even for medium-sized inputs.
- $O(2^n)$ grows exponentially and becomes impractical for large problems because the execution time increases dramatically.

Analysis

From the comparison table, it is evident that algorithms with lower-order complexities scale much better than higher-order algorithms. Constant and logarithmic algorithms require the least computation and remain efficient even when the input size becomes very large. Linear algorithms increase proportionally with input size and are widely used because of their predictable performance. Linearithmic algorithms provide an excellent balance between speed and functionality, making them the preferred choice for sorting operations.

Quadratic and cubic algorithms require significantly more operations as input size grows, making them

unsuitable for large datasets. Exponential algorithms experience explosive growth in execution time, making them practical only for very small inputs or specialized optimization problems.

Efficiency Ranking

Rank	Time Complexity	Performance
1	$O(1)$	Best
2	$O(\log n)$	Excellent
3	$O(n)$	Very Good
4	$O(n \log n)$	Good
5	$O(n^2)$	Average
6	$O(n^3)$	Poor
7	$O(2^n)$	Worst

Advantages of Visualization-Based Analysis

- Simplifies the comparison of multiple algorithms.
- Clearly shows how execution time increases with input size.
- Helps identify algorithms suitable for large datasets.
- Supports algorithm selection based on scalability.
- Improves understanding of theoretical time complexity.

Applications

- Selecting efficient sorting and searching algorithms.
- Performance evaluation in software development.
- Database query optimization.
- Machine learning and data processing.
- Scientific computing and large-scale simulations.

Conclusion

Visualization-based algorithm efficiency analysis provides a simple and effective way to compare the performance of algorithms. By analyzing the growth of time complexity with increasing input size, it becomes clear that $O(1)$, $O(\log n)$, $O(n)$, and $O(n \log n)$ algorithms are highly scalable and efficient for practical applications. In contrast, $O(n^2)$, $O(n^3)$, and $O(2^n)$ algorithms exhibit rapid growth in execution time, making them unsuitable for handling large datasets. Therefore, understanding algorithm complexity and comparing growth trends enables developers to choose the most efficient algorithm, resulting in faster execution, lower resource consumption, and improved overall system performance.

